CSCC24 – Principles of Programming Languages

Anya Tafliovich[1]

[1] with thanks to S.McIlraigh, G.Penn, P.Ragde

motivation

Consider the following Scheme function definition:

```
(define foobar
  (lambda (x)
    (if (even? x)
        (car x)
        (cdr x))))
```

Anything wrong?

We are able to figure it out before run-time. Can't a PL support this kind of reasoning?

motivation

- It could be worse!

- At least Scheme checks that `x` is a pair before accessing memory in the execution of `(cdr x)`.

- In fact, Scheme is <u>type safe</u>: it will never execute `(op arg)` if `op` is not applicable to `arg`.

- Languages which are not type safe (e.g., C) allow unsafe memory accesses: a major source of security vulnerabilities.

motivation

- Want to catch all errors at compile-time.
  - Extremely difficult, if not impossible.
- Identify a certain class of errors and guarantee to catch all of them.
- Catch as many errors as possible?
  - Costly: programmers' expertise, computational resources, etc.
  - Slower development, less expressive languages.
  - Used in development of safety-critical systems.
- Compromise: type systems.
  - Guarantees range widely: from basic safety of memory accesses to just about anything you want.

typing

A <u>type</u> is:

"A name for a set of values and some operations which can be performed on that set of values."

"A collection of computational entities that share some common property."

typing

Some examples of types (in Haskell notation):

- `Int` : the integers
- `Integer` : the integers, unbounded
- `Float, Double` : the real floating point numbers, single and double precision
- `Char` : the characters
- `Bool` : the booleans (`True` and `False`)
- `Int` $\rightarrow$ `Bool` : the functions that take integers as input and return booleans as output.

What constitutes a type is language dependent.

typing

Benefits of having a type system:

- Easier to debug programs: compiler can catch many errors.
- Static analysis: a lot of useful information about the program can be obtained at compile-time.
- Efficiency: typing can be used by the compiler to generate quicker code.
- Correctness: typing can be used (by the programmer or by the compiler) to prove correctness of code.
- Documentation: types declare your intent with well-chosen names.

typing

A programming language is type safe if no program is allowed to violate its type distinctions.

The process of verifying and enforcing the constraints of types is called type checking.

Type checking can either occur at compile-time (static type checking) or at run-time (dynamic type checking).

static vs dynamic typing

Dynamic type checking:

- Performed at run-time.
- Slower execution: need to carry type information around, lots of run-time checks.
- More flexible.
- Easier refactoring.

Static type checking:

- Faster execution.
- Compiler can do a lot of optimization.
- Some argue that resulting programs are safer.
- Some argue that resulting programs are more elegant and modular.
- Some argue that programmers will write horrible code to get around a static type-checker.

static typing

Explicit static typing: code contains type annotations.

- For example, in Java:
  - Variable declarations
    `int x,y,z;`
  - Function headers
    `public static void main(String[] arg)`

Type inference: infer all types from the code that does not contain explicit type annotations.

- `foo (x, y, z) = if x then "hello, " ++ y`
  `else "hi, " ++ z`

Haskell data types

Basic types:

- `()` — the only member is `()`.
- `Bool` — booleans.
- `Int, Integer` — integers.
- `Float, Double` — reals.
- `Char` — characters.

More types:

- $(\langle \texttt{type}_0 \rangle, \langle \texttt{type}_1 \rangle, \ldots, \langle \texttt{type}_n \rangle)$ — tuples.
- $[\langle \texttt{type} \rangle]$—: lists.
- $\langle \texttt{input-type} \rangle \to \langle \texttt{output-type} \rangle$ — functions.

Haskell basic types

(): (called "unit") this type has only one element `()`

```
prompt> ()
()
prompt> :type ()
() :: ()
prompt> :t ()
() :: ()
prompt>
```

Reading the above snippet of the GHCi interpreter session:

- evaluate `()`, please
- what is the type of `()`?
- what is the type of `()` (and I don't like typing)?

Haskell basic types

Bool: this type has two elements: `True` and `False`.

Operations on `Bools`: `not`, `&&`, `||`, `if`, ...

```
prompt> True && False
False
prompt> True || False
True
prompt> not True
False
prompt> if False then 'a' else 'b'
'b'
```

Haskell basic types

Lots of number types (more on this later): `Int`, `Integer`, `Float`, `Double`

```
prompt> 3.14 + 3.14 ** 2 - 42
-29.0004
prompt> 3.14 + 3.14 ** 2 - 42 > -30
True
```

Read the documentation for all that is available for number types.

Haskell basic types

Char: { 'a', 'b', 'c', ... }

Haskell data types

More types:

- $(\langle \texttt{type}_0 \rangle, \langle \texttt{type}_1 \rangle, \ldots, \langle \texttt{type}_n \rangle)$ — tuples.
- $[\langle \texttt{type} \rangle]$ — lists.
- $\langle \texttt{input-type} \rangle \to \langle \texttt{output-type} \rangle$ — functions.

Haskell types

A **tuple** packs together several types.

```
prompt> ('a','b',True)
('a','b',True)
prompt> :t ('a','b',True)
('a','b',True) :: (Char, Char, Bool)
```

Haskell types

In Haskell all elements in a **list** must have the same type.

```
prompt> [1,2,3]
[1,2,3]
prompt> ['a','b','c']
"abc"
prompt> [True,False]
[True,False]

prompt> [[42],[24,3]]
[[42],[24,3]]

prompt> [True,False,"nono"]
<interactive>...:
 Couldn't match expected type 'Bool' with actual type '[Char]'
 In the expression: "nono"
 In the expression: [True, False, "nono"]
 In an equation for 'it': it = [True, False, "nono"]
```

`[]` is the empty list.

Constructor: `:`

More operations: `++`, `null`, `length`, `map`, `foldr`, `...`

Haskell types — list

Some examples:

```
prompt> 1 : [2,3]
[1,2,3]
prompt> [1,2] ++ [2,3]
[1,2,2,3]
prompt> length [1,2]
2
prompt> map abs [1,-2,-3,4]
[1,2,3,4]
prompt> null [1,2]
False
prompt> null []
True
prompt> foldr (+) 0 [1,2,3] # more on this (+) later
6
```

Haskell syntax

A variable declaration in Haskell looks like:

- ⟨`name`⟩ `=` ⟨`expr`⟩
- `x = 42 + 24`
- `(define x (+ 42 24))`

Haskell functions

The syntax for anonymous **functions** in Haskell is:

- `\⟨arg⟩ -> ⟨body⟩`
- `\x -> x + 1`
- `(lambda (x) (+ x 1))`

Giving a name to a function:

- `⟨name⟩ = \⟨arg⟩ -> ⟨body⟩`
- `inc = \x -> x + 1`
- `(define inc (lambda (x) (+ x 1)))`

Or:

- `⟨name⟩ ⟨arg⟩ = ⟨body⟩`
- `inc x = x + 1`
- `(define (inc x) (+ x 1))`

In Haskell every function accepts exactly one argument. This argument could be a tuple.

Haskell types

The **type of a function** is determined by the type of the input and the type of the output.

Some examples:

```
prompt> implies (x, y) = if x then y else True
prompt> :t implies
```

```
prompt> greet x = "hello, " ++ x
prompt> :t greet
```

```
prompt> maybeGreet (g,x) =
            if g then "hello, " ++ x else x
prompt> :t maybeGreet
```

parametric polymorphism

What is the type of `\x -> x`?

```
prompt> :t (\x -> x)
(\x -> x) :: t -> t
```

```
prompt> id =  \x -> x
prompt> id 324
324
prompt> id "foo"
"foo"
prompt> id [1,2,3]
[1,2,3]
prompt> :t (id "foo")
(id "foo") :: [Char]
```

parametric polymorphism

```
prompt> id = \x -> x
prompt> :t id
id :: t -> t
```

`t` is a <u>type variable</u>.

`id` is a <u>polymorphic</u> function.

$\alpha \to \alpha$ means "for every valid type α, $\alpha \to \alpha$"
$\forall \alpha \cdot \alpha \to \alpha$

When `id` is applied to `324`, the type variable α is instantiated to `int`.

parametric polymorphism

Examples:

```
choose (a,b,c) = if a then b else c
```

```
swap (x,y) = (y,x)
```

parametric polymorphism

What is the type of the following function?

```
repeatTwice lst =
  if null lst then lst
  else [head lst,head lst] ++ repeatTwice (tail lst)
```

List is a <u>polymorphic data type</u>.

pattern matching

Value declaration (general form): `<pat> = <exp>`

```
prompt> (foo,bar) = ('a','b')
prompt> foo
'a'
prompt> bar
'b'
prompt> [foo,bar] = ["foo","bar"]
prompt> foo
"foo"
prompt> bar
"bar"
prompt> x : xs = [1,2,3]
prompt> x
1
prompt> xs
[2,3]
```

pattern matching

"_" is "don't care": matches everything, binds nothing

```
prompt> (_,x,y) = (1,2,3)
prompt> x
2
prompt> y
3

prompt> [x,_,z] = [1,2,3]
prompt> x
1
prompt> z
3
```

pattern matching

Function declaration with pattern matching:

```
<name> <pattern1> = <exp1>
<name> <pattern2> = <exp2>
   .
   .
   .
<name> <patternN> = <expN>
```

This means: the function name is `name`. It takes one argument (as any other Haskell function). It tries to match the argument to `pattern1`. If it succeeds, it returns value of `exp1`. Otherwise, tries to match the argument to `pattern2`. Etc, etc.

pattern matching

For example we can (and definitely should!) rewrite `repeatTwice` to use pattern matching:

```
repeatTwice [] = []
repeatTwice (x : xs) = x : x : repeatTwice xs
```

```
len [] = 0
len (x : xs) = 1 + len xs
```

Even better:

```
len [] = 0
len (_ : xs) = 1 + len xs
```

pattern matching

Function `firstlist` takes a list of pairs and returns the list consisting of the first elements only. For example:

```
firstlist [] ==> []
firstlist [(1,2),(1,3)] ==> [1,1]
firstlist [(1,"a"),(2,"b"),(3,"c")] ==> [1,2,3]
firstlist [([],"a"),([1],"b"),([1,2],"c")] ==>
    [[],[1],[1,2]]
```

Haskell notes

```
reverse [] = []
reverse (x : xs) = reverse xs ++ [x]
```

Let:

```
reverse xs =
    let
        rev ([], rs) = rs
        rev (x : xs, rs) = rev (xs, x : rs)
    in rev (xs, [])
```

Haskell notes

Where:

```
reverse xs = rev (xs, []) where
        rev ([], rs) = rs
        rev (x : xs, rs) = rev (xs, x : rs)
```

Guards:

```
abs x = if x >= 0 then x else -x

abs x | x >= 0 = x
      | otherwise = -x
```

type classes

So, what is the type of 42?

```
42 :: Num a => a
```

What does this mean?
`Num` is a <u>type class</u>.

`Num a` is a type <u>class constraint</u>.

`Num a => a` means some type `a` in the class `Num`.

And what is the `Num` class? All types in this class must implement addition, subtraction, multiplication, negation, absolute value, and other. See Haskell documentation for details.

type classes

Type classes offer a controlled approach to overloading.

There are a number of predefined type classes: `Eq`, `Ord`, `Show`, `Read`, `Num`, and more.

You can create instances of these classes.

You can also create your own classes and instantiate them.

(These are not like Python/Java classes. More like Java interfaces.)

type classes

The `Eq` type class is all types with equality defined.

Types in this class provide == and /=.

```
member (_, []) = False
member (y, x : xs) = x == y || member (y, xs)
```

```
prompt> :type member
member :: (Eq t) => (t, [t]) -> Bool
```

`(Eq t)` is a type class constraint.

All the base types (`Int`, `Bool`, etc.) are members of `Eq`.

Much more on type classes later.

currying

- the process of transforming a function that takes multiple arguments in a tuple as its argument, into a function that takes just a single argument and returns another function which accepts further arguments, one by one, that the original function would receive in the rest of that tuple. [HaskellWiki]

- the technique of converting a function that takes multiple arguments into a sequence of functions that each take a single argument. [Wikipedia]

- A function that "takes two arguments and returns the result" is a function that "takes one arguments and returns a function that takes one argument and returns the result".

- `(a,b) -> c` is transformed to `a -> b -> c`

currying

- `(a,b) -> c` is transformed to `a -> b -> c`

- `(a,b,c) -> d` is transformed to `a -> b -> c -> d`

- Notice that `->` is right associative and, accordingly, function application is left-associative. That is,
  - `a -> b -> c` is the same as `a -> (b -> c)` , and
  - `f x y` is the same as `(f x) y`

currying

With named functions:

```
sum (x,y) = x + y          -- not curried version
sum' x y = x + y           -- curried version

prompt> :t sum

sum :: Num a => (a, a) -> a

prompt> :t sum'

sum' :: Num a => a -> a -> a
```

currying

Currying gives us partial application:

```
sum' x y = x + y          -- curried version
sum' :: Num a => a -> a -> a

prompt> sum' 2 3
5

prompt> sum' 2            -- partial application
sum' 2 :: Num a => a -> a

add2 = sum' 2
add2 :: Num a => a -> a

prompt> add2 3
5
```

currying

With anonymous functions:

```
prompt> :t (\x -> \y -> x + y)

(\x -> \y -> x + y) :: Num a => a -> a -> a

prompt> :t (\x -> \y -> x + y) 42

(\x -> \y -> x + y) 42 :: Num a => a -> a
```

currying

Built-in `curry` and `uncurry`:

```
curry :: ((a, b) -> c) -> a -> b -> c
```

```
prompt> member
member :: Eq t => (t, [t]) -> Bool
prompt> curry member
```

```
curry member :: Eq t => t -> [t] -> Bool
prompt> memberC = curry member
prompt> memberC 1 [1, 2, 3]
True
```

currying

Built-in `curry` and `uncurry`:

```
uncurry :: (a -> b -> c) -> (a, b) -> c
```

```
prompt> sum'
sum' :: Num a => a -> a -> a
prompt> uncurry sum'

uncurry sum' :: Num c => (c, c) -> c
prompt> sum'' = uncurry sum'
prompt> sum'' (2, 3)
5
```

functions and operators

Any two-parameter curried function can be used as an operator:

```
prompt> elem 3 [2,3,4]
True
prompt> 3 `elem` [2,3,4]
True
```

Any operator can be used as a function:

```
prompt> (*) 2 3
6
prompt> (:) 2 [3,4]
[2,3,4]
```

sections

```
prompt> map (+2) [1,2,3]

[3,4,5]
prompt> map (2+) [1,2,3]

[3,4,5]
prompt> map (/2) [1,2,3]

[0.5,1.0,1.5]
prompt> map (2/) [1,2,3]

[2.0,1.0,0.6666666666666666]
prompt> map (:[42]) [1,2,3]

[[1,42],[2,42],[3,42]]
prompt> map (42:) [[1],[2,3],[4,5]]

[[42,1],[42,2,3],[42,4,5]]
```

type synonyms

We can give existing types new names. Syntax:

$$\texttt{type}\ \mathit{NewType} = \mathit{OldType}$$

NewType becomes an alias (a synonym) for the **existing** type *OldType*.

```
type String = [Char]

type PhoneNumber = String
type Name = String
type PhoneBook = [(Name,PhoneNumber)]
```

user defined datatypes

General Syntax:

```
data NewType =
        Cons1 Type1
      | Cons2 Type2
        ...
      | ConsN TypeN
```

- Defines a **new** type called `NewType`.
- `Type1,...,TypeN` are previously defined types.
- `Cons1,...,ConsN` are <u>constructors</u>. They are used to create a value of `NewType` type.
- `Type` is omitted if a constructor does not need any argument (such constructors are called <u>constants</u>).

enumerated types

All constructors are constants (no argument).
Example:

```
data Colour = Red | Green | Blue
c = Red

colorName Red = "red"
colorName Green = "green"
colorName Blue = "blue"


prompt> :t c

c :: Colour
prompt> :t colorName

colorName :: Colour -> [Char]
prompt> colorName Blue

"blue"
```

variant types

Create union of different types:

```
data Text = Letter Char | Word [Char]

textLen (Letter _) = 1
textLen (Word w) = length w

prompt> :t textLen

textLen :: Text -> Int
```

recursive types

A datatype can be recursive, of course: e.g. a linked list (ignore the deriving business for now)

```
data LList = Nil | Node (Int, LList) deriving Show

llist = Node (1, Node (2, Node(3, Nil)))

llistLen Nil = 0
llistLen (Node (_,rest)) = 1 + llistLen rest

llistLen :: LList -> Int
```

recursive types

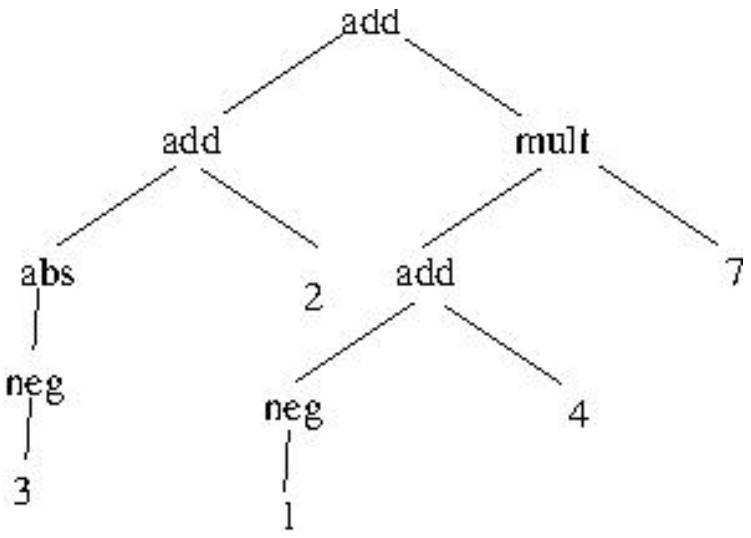
What about a polymorphic linked list?

```
data LList a = Nil | Node (a, LList a) deriving Show

llist1 = Node (1, Node (2, Node(3, Nil)))
llist2 = Node ('1', Node ('2', Node ('3', Nil)))

llistLen Nil = 0
llistLen (Node (_,rest)) = 1 + llistLen rest

llistLen :: LList t -> Int
```

recursive types

Example: Tree representation of simple mathematical expressions.

$$(|-3|+2)+((-1)+4)*7)$$



What is the datatype we need?

```
data MathExpr =
    Leaf Int
  | Unary (Int -> Int, MathExpr)
  | Binary (Int -> Int -> Int, MathExpr, MathExpr)
```

recursive types

The tree in the figure:

```
t = Binary((+),
           Binary((+),
                  Unary(abs,
                        Unary ((0-),
                               Leaf 3)),
                  Leaf 2),
           Binary((*),
                  Binary((+),
                         Unary((0-),
                               Leaf 1),
                         Leaf 4),
                  Leaf 7))
```

recursive types

Evaluating the tree:

```
eval (Leaf v) = v
eval (Unary (f,t)) = f (eval t)
eval (Binary (f,l,r)) = f (eval l) (eval r)
```

Type of eval?

curried type/value constructors

Type/value constructors are simply functions. And so they may be curried.

Value constructor:

```
data BTree a = Empty | Node (a, BTree a, BTree a)
```

`BTree` is a type constructor. `Empty` and `Node` are value constructors.

Curried value constructor:

```
data BTree a = Empty | Node a (BTree a) (BTree a)
```

curried type/value constructors

```
data BTree a = Empty | Node a (BTree a) (BTree a)
```

Example:

```
t = (Node 1 (Node 2 (Node 3 Empty Empty)
                    (Node 4 Empty Empty))
            (Node 5 (Node 6 Empty Empty)
                    (Node 7 Empty
                            (Node 8 Empty Empty))))
prompt> :type t

t :: BTree Integer
```

curried value constructors

```
data BTree a = Empty | Node a (BTree a) (BTree a)

prompt> :t Node "a"

Node "a" :: BTree [Char] -> BTree [Char] -> BTree [Char]

prompt> :t Node "a" Empty

Node "a" Empty :: BTree [Char] -> BTree [Char]

prompt> :t Node "a" Empty (Node "a" Empty Empty)

Node "a" Empty (Node "a" Empty Empty) :: BTree [Char]
```

curried value constructors

```
data BTree a = Empty | Node a (BTree a) (BTree a)

tree2list :: (BTree a) -> [a]
tree2list Empty = []
tree2list (Node v l r) =
    (tree2list l) ++ [v] ++ (tree2list r)
```

A better solution?

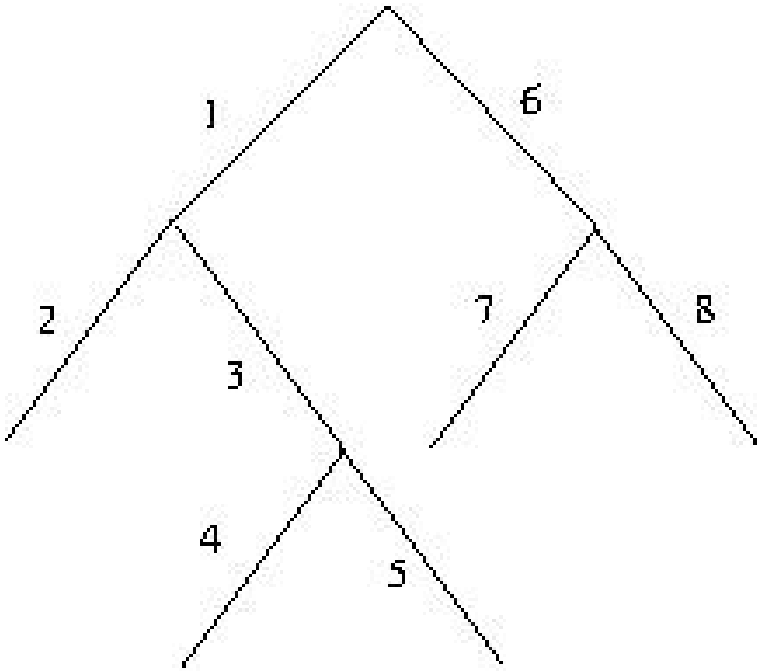
curried value constructors

(Ignore the "deriving" business for now).

```
data Tree a = Leaf a |
              Internal (Tree a) (Tree a) a deriving Show

makeLeafForest = map Leaf
makeForest = map $ Internal (Leaf 42) (Leaf 24)

prompt> makeLeafForest [1,2,3,4,5]

[Leaf 1,Leaf 2,Leaf 3,Leaf 4,Leaf 5]
Prelide> makeForest [1,2,3,4,5]

[Internal (Leaf 42) (Leaf 24) 1,
 Internal (Leaf 42) (Leaf 24) 2,
 Internal (Leaf 42) (Leaf 24) 3,
 Internal (Leaf 42) (Leaf 24) 4,
 Internal (Leaf 42) (Leaf 24) 5]
```

mutually recursive types

Example: a tree with labeled branches:




```
data Tree a = Empty
            | Node (Branch a) (Branch a)
data Branch a = Branch a (Tree a)
```

mutually recursive types

The tree in the figure:

```
lt =
   Node(Branch 1
                (Node (Branch 2 Empty)
                      (Branch 3 (Node (Branch 4 Empty)
                                      (Branch 5 Empty)))))
        (Branch 6
                (Node (Branch 7 Empty)
                      (Branch 8 Empty)))
```

mutually recursive types

```
data Tree a = Empty
            | Node (Branch a) (Branch a)
data Branch a = Branch a (Tree a)
```

Return the list of branch labels, in order:

```
listTree Empty = []
listTree (Node l r) = (listBranch l) ++ (listBranch r)
listBranch (Branch b t) = b : listTree t
```

```
listTree :: Tree a -> [a]
```

```
listBranch :: Branch a -> [a]
```

recursive types

1. A powerful tool for constructing new types.
2. The structure of the datatype suggests the structure of the recursive function on the datatype.

infix value constructors

```
data Rational = Integer :/ Integer deriving Show

r1 = 1 :/ 2
r2 = 1 :/ 3

addRat (x0 :/ y0) (x1 :/ y1) =
    (x0 * y1 + x1 * y0) :/ (y0 * y1)

prompt> r1 `addRat` r2
5 :/ 6
```

Thus `(:) :: a -> [a] -> [a]` is no different.

curried type constructors

```
data Either a b = Left a | Right b
```

```
prompt> :kind Either
Either :: * -> * -> *
```

But let's not go there...

type classes

So, what is the type of 42?

```
42 :: Num a => a
```

What does this mean?
`Num` is a <u>type class</u>.

`Num a` is a type <u>class constraint</u>.

`Num a => a` means some type `a` in the class `Num`.

And what is the `Num` class? All types in this class must implement addition, subtraction, multiplication, negation, absolute value, and other. See Haskell documentation for details.

type classes

Type classes offer a controlled approach to overloading.

There are a number of predefined type classes: `Eq`, `Ord`, `Show`, `Read`, `Num`, and more.

You can create instances of these classes.

You can also create your own classes and instantiate them.

(These are NOT like Python/Java classes. More like Java interfaces.)

type classes

The `Eq` type class is all types with equality defined.

Types in this class provide == and /=.

```
member _ [] = False
member y (x : xs) = x == y || member y xs
```

```
prompt> :type member
```

```
member :: (Eq t) => t -> [t] -> Bool
```

`(Eq t)` is a type class constraint.

All the base types (`Int`, `Bool`, etc.) are members of `Eq`.

Let's look more closely at what type classes are.

type classes

A simple way to create a member of `Eq`:

```
data Btree a = Empty | Node (Btree a) a (Btree a)
               deriving Eq
```

```
prompt> (Node Empty 4 Empty) == (Node Empty 5 Empty)
False
```

```
prompt> (Node Empty 4 Empty) /= (Node Empty 5 Empty)
True
```

```
prompt> (Node Empty 4 Empty) == (Node Empty 4 Empty)
True
```

type classes

We may wish to provide a non-derived equality method.

```
data First = Pair Int Int

instance Eq First where
    (Pair x _) == (Pair y _) = (x == y)

prompt> (Pair 1 3) == (Pair 2 3)

False

prompt> (Pair 1 3) == (Pair 1 4)

True
```

type classes

More general:

```
data First a = Pair a a


instance Eq a => Eq (First a) where
    (Pair x _) == (Pair y _) = (x == y)

prompt> (Pair [1, 2, 3] [1]) == (Pair [1, 2, 3] [3])
True
prompt> (Pair [1] [2]) == (Pair [2] [2])
False
```

type classes

Providing a **default definition** of a function in the type class definition:

```
class Eq a where
    (==), (/=) :: a -> a -> Bool
    x /= y = not (x == y)
    x == y = not (x /= y)
```

Either of these default definitions may be overridden.

```
prompt> (Pair [1, 2, 3] [1]) /= (Pair [1, 2, 3] [3])
False
prompt> (Pair [1] [2]) /= (Pair [2] [2])
True
```

type classes

`Ord` inherits from `Eq` and specifies the four comparison operators <, <=, >, >=. It gives default definitions for `min` and `max` in terms of these.

There is also a three-way `compare` function which returns `LT`, `EQ`, and `GT`.

Most basic datatypes are instances of `Ord`, and user-defined datatypes can derive `Ord` (lexicographic ordering).

type classes

`Show` specifies the method `show :: a -> String`.

`Read` specifies the method `read :: String -> a` and can be used for parsing.

`Num` inherits from `Eq`, and specifies `+`, `-`, `*`, `negate`, `abs`, and `signum`.

Division in handled by `Integral` and `Fractional`, which inherit from `Num`.

Use `:info <typeclass>` too see the instances of a typeclass.

type classes

Defining our own type class:

```
class YesNo a where
    yesno :: a -> Bool
```

and some instances:

```
instance YesNo Integer where
    yesno 0 = False
    yesno _ = True

instance YesNo [a] where
    yesno [] = False
    yesno _ = True
```

type classes

```
instance YesNo Bool where
    yesno x = x
instance YesNo (BTree a) where
    yesno Empty = False
    yesno _ = True
```

Then

```
prompt> yesno []
False
prompt> yesno ""
False
prompt> yesno [1, 2, 3]
True
prompt> yesno "abc"
True
```

type classes

cont.

```
prompt> yesno Empty
False
prompt> yesno (Node "a" Empty Empty)
True
prompt> yesno True
True
prompt> yesno (1 == 0)
False
prompt> yesno 0
False
prompt> yesno 42
True
```

type classes

cont.

```
prompt> :t yesno
yesno :: (YesNo a) => a -> Bool

prompt> :info YesNo
class YesNo a where yesno :: a -> Bool
   -- Defined at /...path.../filename.hs:140:6-10
instance YesNo Integer
  -- Defined at /...path.../filename.hs:(143,0)-(145,17)
instance YesNo [a]
  -- Defined at /...path.../filename.hs:(147,0)-(149,17)
instance YesNo Bool
  -- Defined at /...path.../filename.hs:(151,0)-(152,19)
instance YesNo (BTree a)
  -- Defined at /...path.../filename.hs:(154,0)-(156,17)
```

list comprehensions

```
map::  (a -> b) -> [a] -> [b]
map f xs = [f x | x <- xs]

filter::  (a -> Bool) -> [a] -> [a]
filter p xs = [x | x <- xs, p x]

cross::  [a] -> [b] -> [(a,b)]
cross xs ys = [(x,y) | x <- xs, y <- ys]

quicksort ::  Ord a => [a] -> [a]

quicksort  []     =  []
quicksort (x:xs) =  quicksort [y | y <- xs, y < x ]
                    ++ [x]
                    ++ quicksort [y | y <- xs, y >= x]
```

list comprehensions

"Python's list comprehension syntax is taken (with trivial keyword/symbol modifications) directly from Haskell. The idea was just too good to pass up." `wiki.python.org`

```
>>> [x + 42 for x in range(5)]
[42, 43, 44, 45, 46]

>>> [(x, y) for x in [1,2,3] for y in [3,1,4] if x != y]
[(1, 3), (1, 4), (2, 3), (2, 1), (2, 4), (3, 1), (3, 4)]
```

playing with Haskell

Ranges:

```
prompt> [1..10]
[1,2,3,4,5,6,7,8,9,10]
prompt> [1,3..10]
[1,3,5,7,9]
prompt> [10,9..1]
[10,9,8,7,6,5,4,3,2,1]
prompt> [10,8..1]
[10,8,6,4,2]
```

lazy evaluation

Don't evaluate before you have to:

```
and' :: Bool -> Bool -> Bool
and' False _ = False
and' _ x = x
```

```
prompt> head []
```

```
*** Exception: prompt.head: empty list
prompt> and' False (head [] == 2)
```

```
False
```

How did this work?

lazy evaluation

- Expressions are not evaluated when they are bound to variables, but their evaluation is **deferred** until their results are needed by other computations.

- Arguments are not evaluated before they are passed to a function, but only when their values are actually used.

- A **thunk** is an unevaluated value with a recipe that explains how to evaluate it.

- It is possible to **partially evaluate** an expression, for example `( thunk, thunk )`.

- Aside: lookup Racket's `thunk` function.

lazy evaluation

```
length [] = 0
length (_:xs) = 1 + length xs
```

What do we need to evaluate in the expression
`length [42^1234, 42^2345, 42^3456]`?

```
length thunk{[42^1234, 42^2345, 42^3456]}  pattern match
length (_ : thunk{[42^2345, 42^3456]})     function body
1 + length thunk{[42^2345, 42^3456]}       pattern match
1 + length (_ : thunk{[42^3456]})          function body
1 + (1 + length thunk{[42^3456]})          pattern match
1 + (1 + length (_ : thunk{[]}))           function body
1 + (1 + (1 + length thunk{[]}))           pattern match
1 + (1 + (1 + length []))                  function body
1 + (1 + (1 + 0))                          display
3
```

lazy evaluation

```
ones = 1 : ones
```

What kind of a thing is `ones`?
What happens if we evaluate `ones` in the REPL?
But we can use it in other ways. For example:

```
prompt> tenOnes = take 10 ones
[1,1,1,1,1,1,1,1,1,1]
```

How does this work? We need to know how `take` is defined:

```
take _ [] = []
take 0 _ = []
take n (x:xs) = x : take (n-1) xs
```

lazy evaluation

```
take _ [] = []
take 0 _ = []
take n (x:xs) = x : take (n-1) xs
```

Then:

```
take 3 thunk{ones}                          pattern match
take 3 (1 : thunk{ones})                    function body
1: take thunk{3-1} thunk{ones}              pattern match
1: take 2 (1 : thunk{ones})                 function body
1:1 : take thunk{2-1} thunk{ones}           pattern match
1:1 : take 1 (1 : thunk{ones})              function body
1:1:1 : take thunk{1-1} thunk{ones}         pattern match
1:1:1 : take 0 _                            function body
1:1:1: []
```

lazy evaluation

More fun examples:

```
numsFrom n = n : numsFrom (n + 1)

nats = numsFrom 0
nats = 0 : map (+1) nats

squares = map (^2) nats

odds = filter odd nats
evens = filter even nats

fibs = 0 : 1 : zipWith (+) fibs (tail fibs)
fibs = 0:1:[x+y | (x, y) <- zip fibs (tail fibs)]

prime x = null [y | y <- [2..(x-1)], x `mod` y == 0]
primes = [x | x <- numsFrom 2, prime x]
primes = filter prime $ numsFrom 2
```